

PROJECT: AWS Automated Backup and Restore System

Services: Amazon EC2, Amazon EBS, AWS Lambda, Amazon EventBridge, AWS IAM, Amazon CloudWatch

Project Goal: The goal of this project is to automatically create daily backups of EC2 EBS volumes to prevent data loss and improve recovery without manual effort.

Skills: AWS EC2, EBS Snapshots, AWS Lambda, EventBridge Scheduling, IAM Permissions, CloudWatch Monitoring, Serverless Automation

Definition: This project implements an automated backup system for Amazon EBS volumes by using AWS Lambda triggered through Amazon EventBridge scheduling, ensuring regular, reliable, and scalable backups of EC2 storage without manual effort, thereby improving data protection and disaster recovery readiness.

Scope of the Project: The scope of this project includes automating the backup of Amazon EBS volumes attached to EC2 instances using a serverless, event-driven approach. It focuses on scheduled snapshot creation through AWS Lambda and Amazon EventBridge, ensuring only tagged volumes are backed up. The project reduces manual effort, minimizes the risk of data loss, improves disaster recovery readiness, and provides a scalable and cost-effective solution suitable for real-world cloud environments..

Methodologies:

The same AWS Automated Backup and Restore System can be implemented using the following methods:

1. Manual EBS Snapshot Creation

In this method, snapshots are created manually using the AWS Management Console or CLI. While it is simple to implement, it is highly error-prone, time-consuming, and not suitable for large-scale or long-term backup requirements.

2. EC2-Based Cron Jobs

This approach uses scheduled cron jobs running on an EC2 instance to trigger snapshot creation scripts. Although it automates backups, it requires continuous server maintenance, incurs additional costs, and introduces a single point of failure if the instance goes down.

3. Third-Party Backup Tools

Third-party tools can manage backups efficiently but add extra licensing costs, dependency on external vendors, and may raise security and compliance concerns.

Chosen Method: Serverless Event-Driven Architecture

The selected methodology uses AWS Lambda triggered by Amazon EventBridge. This approach is more reliable because it is fully serverless, eliminating infrastructure management. It automatically scales, ensures consistent execution through AWS-managed scheduling, reduces operational overhead, and leverages native AWS security and monitoring services. As a result, it provides a cost-effective, scalable, and production-ready solution compared to other methods.

Services Used in this Project:

1. AWS Lambda

Definition:

AWS Lambda is a serverless computing service that runs code without the need to provision or manage servers.

Purpose:

It is used to automatically execute backup logic based on scheduled events.

Role in This Project:

AWS Lambda identifies EBS volumes with specific tags and creates automated snapshots without manual intervention.

2. Amazon EC2

Definition:

Amazon EC2 provides scalable virtual computing resources in the cloud.

Purpose:

It hosts applications and workloads that require persistent storage.

Role in This Project:

EC2 instances use EBS volumes that are periodically backed up through automated snapshots.

3. Amazon EBS (Elastic Block Store)

Definition:

Amazon EBS is a block-level storage service designed for use with EC2 instances.

Purpose:

It provides persistent storage for EC2 instances.

Role in This Project:

EBS volumes store critical data, and snapshots of these volumes are created as backups.

4. Amazon EventBridge

Definition:

Amazon EventBridge is a serverless event bus service used to connect applications using events.

Purpose:

It enables scheduled and event-driven automation.

Role in This Project:

EventBridge triggers the Lambda function at fixed intervals (daily) to initiate automated backups.

5. AWS IAM (Identity and Access Management)

Definition:

AWS IAM is a service that manages secure access to AWS resources.

Purpose:

It controls permissions and access policies.

Role in This Project:

IAM roles and policies allow Lambda to access EC2, EBS, and CloudWatch securely.

6. Amazon CloudWatch

Definition:

Amazon CloudWatch is a monitoring and logging service for AWS resources.

Purpose:

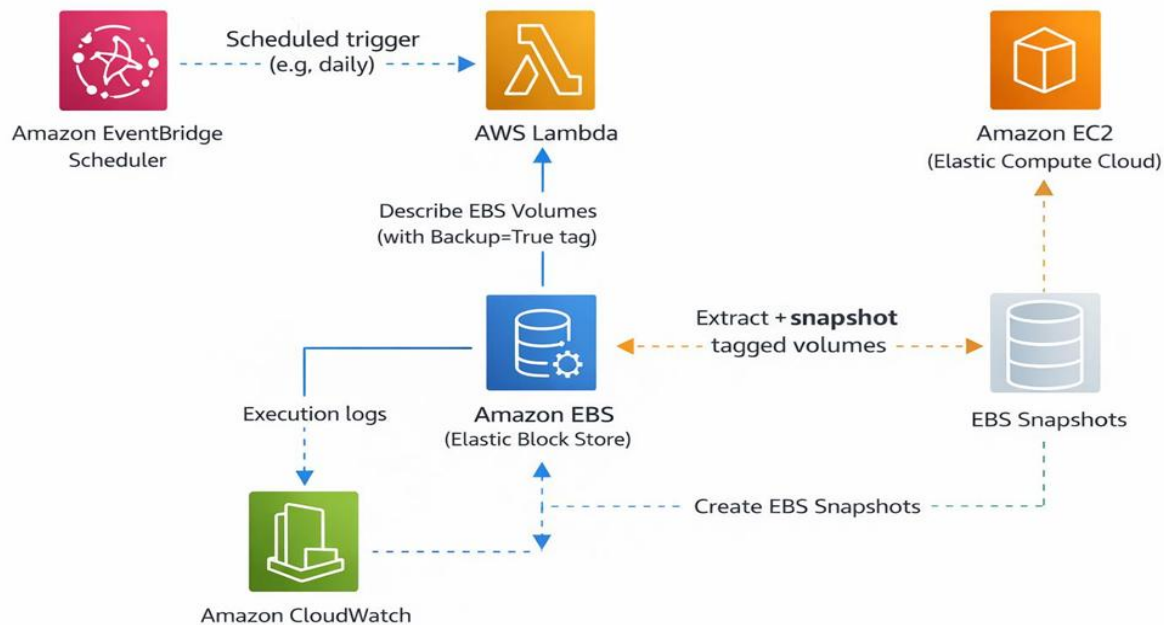
It helps monitor application performance and system health.

Role in This Project:

CloudWatch logs Lambda execution details and helps track backup success or failures.

Architecture Diagram

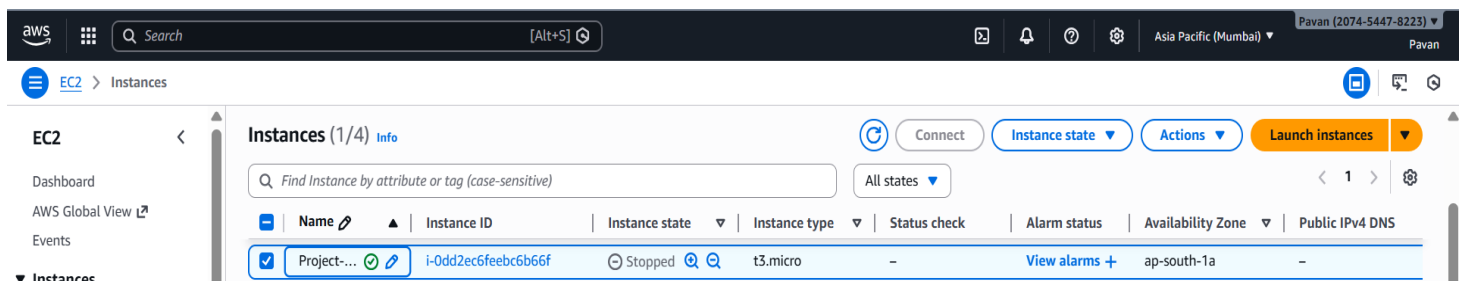
AWS Automated EBS Snapshot Backup Using Lambda & EventBridge



Implementation and Execution of AWS Automated Backup and Restore System .

Phase 1. Create EC-2 Instance

1. Login to **AWS Management Console**
2. Go to **EC2** → **Instances** → **Launch instance**
3. Enter **Instance name** (example: Backup-Test-EC2)
4. Choose **Amazon Linux 2 AMI**
5. Select instance type: **t3.micro** (Free Tier)
6. Create or select a **Key Pair**
7. Click **Launch Instance**



Phase 2: Attach EBS Volume

1. Go to **EC2** → **Volumes**
2. Click **Create volume**
3. Select:
 - Volume type: gp3
 - Size: 5-10 GB
 - Availability Zone: **same as EC2**
4. Click **Create volume**
5. Select the volume → **Actions** → **Attach volume**
6. Choose your EC2 instance

The screenshot shows the AWS Management Console interface for an EBS volume. The left sidebar contains navigation links for various AWS services. The main content area displays the details for volume 'vol-090a596bbb8dd0d41'. The details are organized into a grid with the following information:

Details			
Volume ID vol-090a596bbb8dd0d41	Size 8 GiB	Type gp3	Status check ✓ Okay
AWS Compute Optimizer finding ① Opt-in to AWS Compute Optimizer for recommendations. Learn more	Volume state ✓ In-use	IOPS 3000	Throughput 125
Fast snapshot restored No	Availability Zone aps1-az1 (ap-south-1a)	Created Sun Jan 18 2026 12:43:34 GMT+0530 (India Standard Time)	Multi-Attach enabled No
Attached resources i-0a196f4c7625c95a8 (jen-server): /dev/xvda (attached)	Outposts ARN -	Managed false	Operator -

Phase 3: Add Tag Backup=True

1. Select the **EBS Volume**
2. Go to **Tags** → **Manage tags**
3. Add:
 - **Key:** Backup
 - **Value:** True
4. Save

The screenshot shows the 'Manage tags' interface in the AWS Management Console. The interface includes a description of tags and a form to add a new tag. The 'Key' field is populated with 'Backup' and the 'Value' field is populated with 'True'. The 'Add new tag' button is located at the bottom left of the form. The 'Remove' button is located at the bottom right of the form.

Phase 4 : Create IAM Role for Lambda

1. Go to **IAM** → **Roles** → **Create role**
2. Select **Trusted entity**: AWS service
3. Choose **Lambda**
4. Attach permissions:
 - o AmazonEC2FullAccess
 - o CloudWatchLogsFullAccess
5. Role name: LambdaEBSBackupRole
6. Click **Create role**

The screenshot shows the AWS IAM console interface. The left sidebar contains the 'Identity and Access Management (IAM)' menu with options like Dashboard, Access Management, User groups, Users, Roles, Policies, Identity providers, and Account settings. The main content area is titled 'lambda-ebs-backup-role' and has tabs for 'Permissions', 'Trust relationships', 'Tags', 'Last Accessed', and 'Revoke sessions'. The 'Permissions' tab is active, showing 'Permissions policies (2)' with a search bar and a 'Filter by Type' dropdown set to 'All types'. Below this is a table listing the attached policies:

Policy name	Type	Attached entities
AmazonEC2FullAccess	AWS managed	3
CloudWatchFullAccess	AWS managed	1

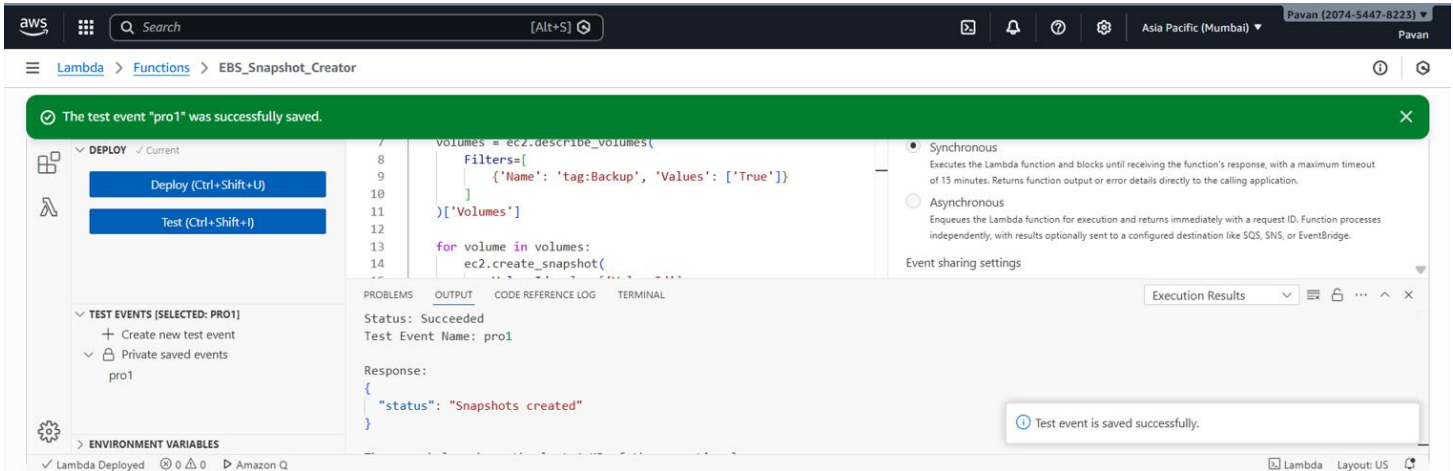
Phase 5 : Create Lambda Function

1. Go to **AWS Lambda** → **Create function**
2. Select **Author from scratch**
3. Function name: ImageprocessorLambda
4. Runtime: **Python 3.10**
5. Execution role: **Use existing role**
6. Select LambdaEBSBackupRole
7. Click **Create function**

The screenshot shows the AWS Lambda console interface. The left sidebar contains the 'Lambda' menu with options like Functions, Layers, and Aliases. The main content area is titled 'ImageProcessorLambda' and has tabs for 'Function overview', 'Layers', and 'Aliases'. The 'Function overview' tab is active, showing a 'Diagram' and 'Template' view. Below this is a card for 'ImageProcessorLambda' with a 'Layers' section showing '(0)' layers. To the right of the card are buttons for 'Add trigger' and 'Add destination'. On the far right, there is a 'Description' section with fields for 'Last modified' (7 seconds ago), 'Function ARN' (arn:aws:lambda:ap-south-1:207454478223:function:ImageProcessorLambda), and 'Function URL'.

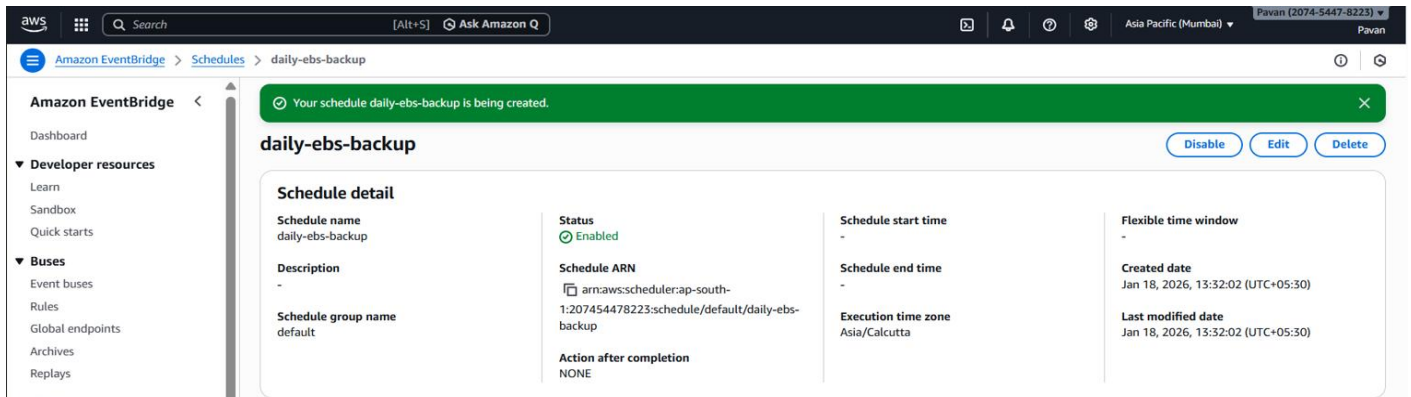
Phase 6: Test Lambda Manually

1. In Lambda console → **Test**
2. Create test event:
 - Event name: TestSnapshot
 - Use default JSON
3. Click **Test**
4. Check:
 - Status: **Succeeded**
 - No error messages



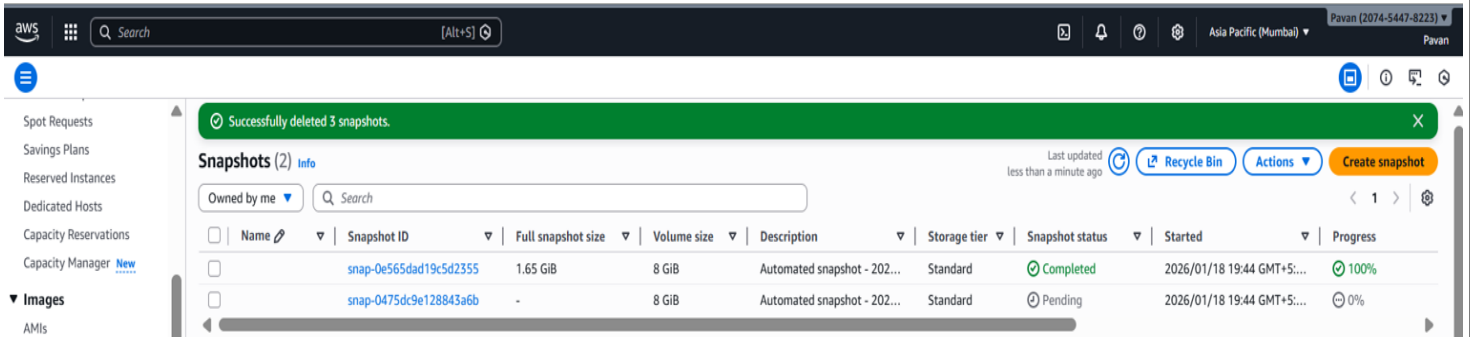
Phase 7 : Configure EventBridge Schedule

1. Go to **Amazon EventBridge**
2. Click **Schedules** → **Create schedule**
3. Name: Daily-EBS-Backup
4. Schedule type: **Rate-based**
5. Set: Rate → 1 day
6. Target:
 - Service: **AWS Lambda**
 - Function: `IamgeprocessorLambda`
7. Role: Create new role (auto)
8. Click **Create schedule**



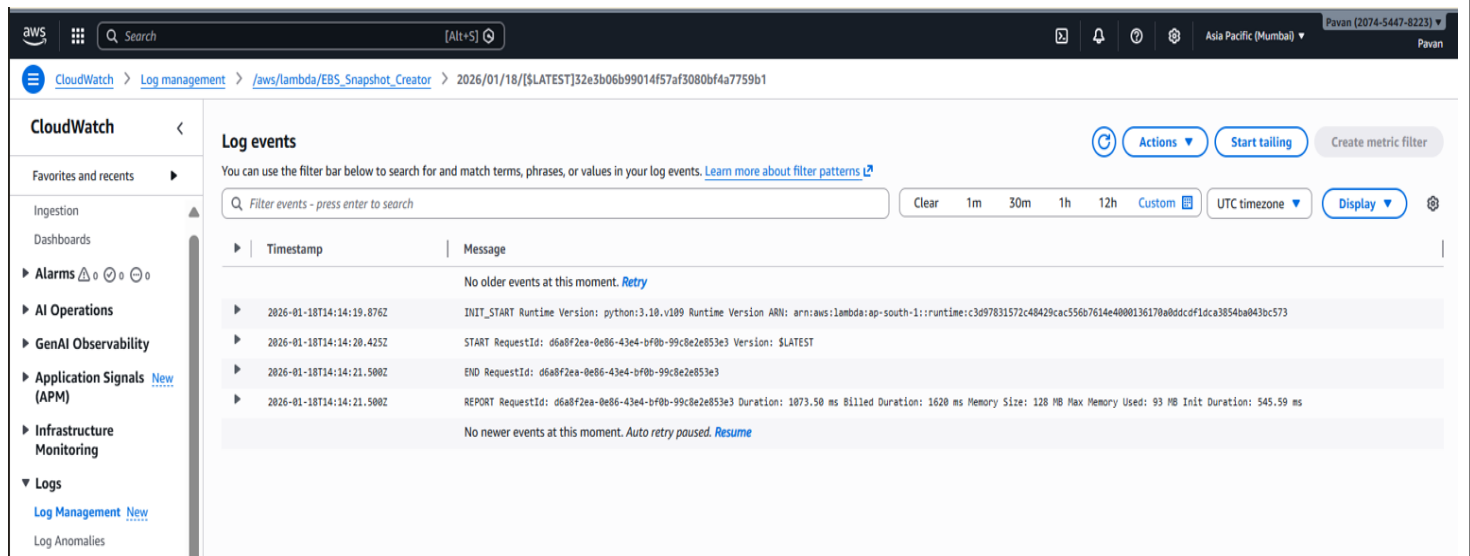
Phase 8 : Verify Snapshots in EC2

1. Go to **EC2** → **Snapshots**
2. Check:
 - Snapshot created
 - Description shows automated backup
 - Correct volume ID



Phase 9: Monitor Logs in CloudWatch

1. Go to **CloudWatch** → **Log groups**
2. Select `/aws/lambda/EBS_Snapshot_Automation`
3. Open latest log stream
4. Verify:
 - No permission errors
 - Snapshot creation logs



TroubleShooting

- **Issue 1: UnauthorizedOperation**

Cause: Lambda role lacked EC2 permissions

Solution: Attached `AmazonEC2FullAccess`

- **Issue 2: Snapshots not created**

Cause: Tag name mismatch (`backup` vs `Backup`)

Solution: Used exact key-value `Backup=True`

Conclusion

This project effectively demonstrates the implementation of an automated backup and recovery system for Amazon EBS volumes using AWS serverless technologies. By integrating AWS Lambda with Amazon EventBridge, the solution enables scheduled, reliable, and hands-free creation of EBS snapshots. The use of IAM ensures secure access control, while Amazon CloudWatch provides continuous monitoring and logging for troubleshooting and performance analysis. Overall, the project reduces operational effort, minimizes human error, improves data protection, and aligns with real-world cloud computing